ChatGPT

Cloudflare-Native Deployment Plan for *A Whittle Wandering* Road Trip App

Architecture Overview

Cloudflare Pages (Frontend): The React/TypeScript single-page app will be deployed on Cloudflare Pages, serving the user interface from a global CDN edge. This provides static hosting for the **AWhittleWandering** site (e.g. **awhittlewandering.com**) [1]. The frontend will remain a static build (Vite + React) served via Pages.

Cloudflare Workers (Backend API): All backend logic will run on Cloudflare Workers at the edge [1]. We'll deploy an API Worker (using the Hono framework) at a custom domain (e.g. **api.awhittlewandering.com** or **admin-api.awhittlewandering.com**). This Worker will handle REST endpoints (under `/api/v1/*`) for data aggregation and admin operations. By using Workers, we get globally low-latency API responses and can easily integrate with Cloudflare's KV, R2, and optional D1.

Cloudflare KV (Caching & State): A Workers KV namespace will be used for temporary caching and lightweight persistence. KV is ideal for storing cached responses (e.g. the last unified data payload), user session tokens, or small metadata like the list of visited states or media metadata. KV provides global replication and low-latency reads, which will help offload repetitive data requests (like frequent Tessie API polls).

Cloudflare R2 (Media Storage): We will create an R2 bucket to store uploaded photos/videos from the road trip (Phase 4 media). The backend Worker will write files to R2 and serve them on request. R2 is an S3-compatible object store, ideal for storing images and other binary media. We'll keep media metadata (like title, location, timestamp) in a persistent store (KV or D1) while the binary content lives in R2 [2]. This separation allows efficient queries on metadata and secure media delivery (by controlling URLs or via Worker).

Cloudflare D1 (Optional Structured DB): If we need structured querying or more complex persistence (for example, a table of timeline events, states visited, or media metadata with filtering), we can integrate Cloudflare D1 (a lightweight SQLite database at the edge). D1 can store trip state (e.g. a table of each state visited with timestamps, or a running log of drives/charges) if KV becomes too limited. For initial deployment, D1 is optional – we'll attempt to use KV + external APIs first – but we design with D1 in mind for future extensibility.

Integration Flow: When a user opens the site, Cloudflare Pages serves the frontend, which in turn calls the Cloudflare Worker API. The Worker will fetch live data from external APIs like Tessie (using secrets) or Mapbox as needed, cache results in KV, and return a unified JSON to the frontend. For media, admin users upload images via the API Worker, which stores files in R2 and records metadata in KV/D1. The images can then be accessed (with proper authorization) via a Worker proxy route or signed URLs. This architecture

keeps all server-side logic on Cloudflare's edge (no separate servers), ensures global low latency, and leverages Cloudflare's security features (WAF, rate limiting, etc.) automatically.

Unified Data API Endpoint (`GET /api/v1/unified-data`)

The `/api/v1/unified-data` endpoint will serve as the **canonical aggregated data API** for both the public dashboard and the admin panel. This single endpoint provides all necessary journey data in one JSON payload, simplifying the frontend logic (one fetch for everything). It replaces or consolidates older endpoints (like separate `/timeline` or `/trip/status` calls) into one unified response.

Route Behavior & Data Aggregation

When the Worker receives a request to `/api/v1/unified-data`, it will perform the following steps:

- **Tessie API Fetch:** Securely call the Tessie API to retrieve the latest vehicle telemetry and trip data. Specifically, the Worker will use the Tessie API Key (stored as a secret in the Worker's environment) to:
- Get the **current vehicle state** (battery level, range, charging status, GPS location, speed, odometer, etc.) via Tessie's `/state` endpoint [3] [4]. This provides real-time telemetry needed for the "currentStatus" field.
- (Optionally) Fetch recent **driving and charging history** via Tessie's `/drives` and `/charges` endpoints for the trip's timeframe [5] [6]. This can be used to build or update the timeline of the journey.

- Check Tessie API connectivity status – if the Tessie API or vehicle is unreachable (vehicle asleep or network issues), note this for the `tessieStatus`.

- **Data Combination:** Merge the Tessie data with any additional state to form a unified response object. The unified payload will be structured as:

```
{
  "journey": {
    "overview": { ... },
    "currentStatus": { ... },
    "timeline": [ ... ],
    "liveData": { ... },
    "tessieStatus": { ... }
  }
}
```

Each sub-object is populated as follows: - `overview`**:** High-level trip stats and progress. This includes fields such as: - **Trip name** (e.g. "48 Continental States Tesla Adventure"). - **startDate** and **currentDate** (trip start and now). - **daysElapsed** (days since start) – this can be computed by comparing start date and today [7]. - **statesVisited** and **statesRemaining** – number of U.S. states visited so far vs remaining (target is 48). We can determine this by analyzing the route or using "state detection" logic on the GPS data [8]. For example, by parsing Tessie's location addresses or coordinates of drives, the Worker can accumulate a set of states

visited and count them. - **progressPercentage** – e.g. 60% if 29/48 states done [9]. - **totalMiles** and **averageMilesPerDay** – total miles driven on the trip and daily average. The Worker can compute these from Tessie data: e.g. subtract starting odometer from current odometer for total trip miles, or sum all Tessie drive distances [10] [11]. Average miles/day = totalMiles / daysElapsed.

```
*Example:* In development, these values were hard-coded for testing (e.g. 29 states visited, 11,950 miles, 56 days) [12] [13]. In production, the Worker will compute them dynamically from real data.
```

- `currentStatus`**:** Real-time telemetry and location of the vehicle. This includes:
  - **Location** – state, city, and coordinates (latitude/longitude) of the car's last known position [14]. Tessie's state endpoint provides GPS lat/long and possibly a resolved address; the Worker can reverse-geocode coordinates to city/state via Mapbox if needed, or parse Tessie's address fields.
  - **Vehicle** – key vehicle metrics like batteryLevel (%), batteryRange (est. miles remaining), chargingState (e.g. `"Charging"` or `"Disconnected"`), odometer (total miles on car), speed (if moving), outsideTemp, etc [15]. These come directly from Tessie's `/state` JSON (charge_state, drive_state, climate_state, etc.) [16] [4], mapped to our schema.
  - **lastUpdate** – timestamp of when this data was last fetched/updated.

  This section corresponds to the **CurrentStatus** interface on the frontend [14] [17]. For example, it might look like:

```
"currentStatus": {
  "location": { "state": "Connecticut", "city": "Norwalk", "coordinates": { "lat": 41.1, "lng": -73.4 } },
  "vehicle": { "batteryLevel": 82, "batteryRange": 267.5, "chargingState": "Complete", "odometer": 70128.5, "speed": 0, "outsideTemp": 25.5 },
  "lastUpdate": "2025-07-31T23:45:00Z"
}
```

  indicating the car's last known state [16] [4].

- `timeline`**:** The chronological **timeline of the journey** (drives, charges, and notable events). This is likely an array of entries, where each entry could represent a completed drive segment or charge session. We can populate this by transforming Tessie's historical data:
  - Use Tessie's `/drives` endpoint to get all drives from the trip start to now [5] [18]. Each drive can be transformed into a simpler object (start time, end time, start location, end location, distance, battery usage, etc.) [10] [19].
  - Use Tessie's `/charges` endpoint similarly for charging events [6] [20].
  - The timeline array can combine drives and charges sorted by time, creating a full story of the trip. For example, a drive entry might include start/end points and miles, while a charge entry includes location and energy added.

    - *Performance:* These historical calls can be heavy. We will **cache the timeline** so that we don't fetch the entire history on every request. The Worker can store the timeline data in KV after the first fetch. On subsequent calls to `/unified-data`, if the timeline is already cached and up-to-date (no new drives since last fetch), we return the cached timeline (saving API calls and latency). If a new day's drive has completed, the Worker can fetch just the new data (using Tessie's `from` timestamp parameter to get recent drives since the last one) and append to the cached timeline.

- `liveData`**:** This field can hold **real-time streaming data points** if needed (e.g. intermediate telemetry for the ongoing drive). In the current design it may remain `null` or empty, but the infrastructure is there to support more granular live telemetry if we push data in real-time (for instance, via a separate process or WebSocket). For now, we might leave `liveData` as an empty object or use it to represent the current in-progress drive (e.g. partial route coordinates if the car is mid-drive). The frontend hook expects this field (even if just for future use) [21].

- `tessieStatus`**:** Metadata about the Tessie integration, indicating if live data is flowing:

    - **connected:** boolean indicating if the Worker successfully fetched data from Tessie recently (e.g. the vehicle is online).
    - **lastSync:** timestamp of the last successful data sync from Tessie.
    - **refreshInterval:** how often the data updates (in seconds). We plan a 30-second interval for polling [22], so this might be `30` or `60` seconds.
    - If Tessie's API is failing or the car is asleep, `connected` may be set to false and lastSync might reflect an older time, signaling to the UI that data is stale.

  This corresponds to the **tessieStatus** object on the frontend [21]. The Worker will set `connected=true` whenever a Tessie API call succeeds. In case of failure, it can set `connected=false` and perhaps reuse the last known data from cache.

- **Caching and Rate Limiting:** To avoid hitting Tessie's API too frequently (and to minimize response time), the Worker will implement caching:

- A KV store (e.g. `AWandering_KV`) can cache the entire `unified-data` JSON payload for a short time (e.g. 15-30 seconds) for **public** data. Since the frontend already polls every 30s [22], caching the Worker's response for ~30s ensures that if multiple clients request data around the same time, Tessie is not called more than once per interval. The Worker can check a timestamp in KV (`last_fetch_time`) to decide if it needs to call Tessie or can serve cached data.
- The timeline portion can be cached much longer (several minutes or hours) because historical data changes infrequently (only when a new drive/charge is added). We might update the timeline cache once per hour or on a trigger (e.g. admin triggers a refresh, or the Worker detects a jump in odometer indicating a new drive ended).
- Cloudflare KV is globally distributed, so all edge instances can share this cache [2]. We will design cache keys like `unified_data_cache` and `timeline_cache` to store JSON blobs.

Additionally, we will configure **rate limiting** at the Worker level to protect the API. Using Cloudflare's built-in tools or a simple in-memory counter, we ensure a single client cannot spam the endpoint. For instance, limit to e.g. 2 requests per 10 seconds per IP (the normal app will only call 1 per 30s). The project already

envisions “production-ready rate limiting” [23], so we can implement a middleware to drop or throttle excessive calls with a 429 response. This protects our Tessie API quota and prevents abuse.

- **Security & Secrets:** The Tessie API key will **never be exposed to the client**. All Tessie calls happen on the server side within the Worker. We’ll store the Tessie API key as a secure environment variable in Cloudflare Workers (using Wrangler secrets or the Cloudflare dashboard). The Worker will inject this key in the Authorization header when calling Tessie [24]. This way, the frontend simply hits our `/unified-data` endpoint with no knowledge of the Tessie credential. Similarly, if Mapbox’s Directions API or any other third-party needs to be called from the server (for example, to decode coordinates to state names or to get route geometry), those API tokens will be kept in the Worker’s config as needed. The client only receives processed data, never raw secrets.

- **Error Handling & Fallback:** The Worker should handle API errors gracefully:

- If a Tessie API call fails (network error or rate limit), the Worker can fall back to cached data. For example, if the last good data is in KV, serve that along with `tessieStatus.connected=false` to indicate it’s stale. This ensures the frontend isn’t left empty-handed if Tessie is briefly unreachable.
- If no cache is available (e.g. first call and Tessie is down), the Worker can return an error response or a minimal stub (perhaps an empty journey with an error flag). We will set appropriate HTTP status (500) or include an `"error"` field in the JSON to signal the front end.
- On the frontend, the `useUnifiedJourneyData` hook already catches errors and stores an error message state [25]. The UI can display a friendly error (e.g. “Failed to fetch live data”) if `error` is set, instead of breaking the app. We will ensure this is in place so that any backend issues are visible to the user/admin.
- We’ll also include a **health-check endpoint** (`GET /health`) in the Worker (already present in code [26]) that returns a simple status. New contributors or monitoring systems can hit `/health` to verify the Worker is running and not misconfigured.

In summary, `GET /api/v1/unified-data` is the centerpiece of the new backend. It securely aggregates **live telemetry, trip progress stats, timeline history, and connection status** into one response. The frontend (both public view and admin view) will use this endpoint exclusively to get all data it needs. This simplifies security (we can harden and test one endpoint) and allows consistent caching and rate-limit strategies for all data fetches.

Media Upload & Storage Endpoints (Cloudflare R2 Integration)

Phase 4 introduced an admin media management feature, which we will now implement using Cloudflare R2 for storage. We will set up dedicated API routes for media handling, ensuring only authenticated admin users can upload or modify media. The two primary endpoints to support are:

- `POST /api/v1/media/upload` – Accepts an image/video file upload (from the admin UI) and stores it to R2.
- `GET /media/:key` – Serves the media content (image or video) by key, possibly with access control.

(Note: The project docs list additional endpoints like `GET /api/v1/media/list` *for listing media, and* `PUT/DELETE /api/v1/media/:id` *for editing and deleting* [27]*. We will configure those as needed, but the upload and delivery endpoints are the focus for Phase 4 deployment.)*

R2 Bucket Configuration

Create a Cloudflare R2 bucket (e.g. named **awhittlewandering-media**). In the Worker's `wrangler.toml`, bind this bucket so the Worker can directly put/get objects. For example:

```
# in wrangler.toml
[r2_buckets]
binding = "MEDIA_BUCKET"      # accessible in code as env.MEDIA_BUCKET
bucket_name = "awhittlewandering-media"
```

This binding allows the Worker to call `env.MEDIA_BUCKET.put(objectName, data)` and `env.MEDIA_BUCKET.get(objectName)` for storing and retrieving files. The R2 bucket will be private (not publicly exposed), so all access goes through our Worker (or via signed URLs we generate). We will also enable appropriate CORS on the bucket if needed (R2 allows configuring CORS rules) so that our Worker or client can fetch objects if direct access is ever used.

`POST /api/v1/media/upload` – File Upload

The admin frontend will call this endpoint when an admin user selects a photo/video to upload. We expect a `multipart/form-data` request containing the file and associated metadata. For example, the request might include fields: - `file` – the binary file content (photo or video). - `location` – a JSON string or fields describing where this media was taken (state, coordinates, etc.), if provided by the UI. - Optionally, `title`, `description`, or other metadata (or these can be added via a separate edit call).

The Worker will handle this as follows: 1. **Auth Check:** Verify the request is from an authenticated admin. We will likely have an admin login system (JWT or session token) already in place for the admin UI. The Worker should check an auth token (e.g. an `Authorization: Bearer <jwt>` header or a session cookie). If the user is not authorized as admin, return HTTP 401. This ensures only the admin can upload media.

1. **Parse Form Data:** Use the Cloudflare Workers API to parse the incoming multipart form data. We'll extract the file (as a `ReadableStream` or `ArrayBuffer`) and any metadata fields. The content type and filename can be obtained from the form data as well.

2. **Generate Object Key:** Create a unique key/name for the file in R2. This could be a UUID or a combination of timestamp and filename. For example, we might generate an ID like `photo-<uuid>.jpg`. The key should be globally unique to avoid collisions. We might also use subfolders or prefixes (e.g. `photos/<id>` vs `videos/<id>`) if we want, but not strictly necessary.

3. **Store in R2:** Put the file into the R2 bucket under the generated key. We'll include the content type metadata so that it's stored correctly. For example:

```
await env.MEDIA_BUCKET.put(objectKey, fileStream, {
    httpMetadata: { contentType: fileType },
    customMetadata: { originalName: filename }
});
```

This will stream the file into the object storage.

4. **Save Metadata:** Record the metadata of this media in a persistent store. We want to retain info like:

5. `id` (the key or a short ID for reference),
6. `name` or original filename,
7. `type` (image/jpeg, video/mp4, etc),
8. `size` (file size),
9. location info (state, coordinates) if provided,
10. upload timestamp,
11. any additional tags or description.

We have two options for storing metadata: - **Cloudflare KV:** We could store a JSON blob in KV keyed by the media ID, or maintain a single KV key that holds an array of all media metadata. KV is simple, but listing or filtering can be tricky (we'd have to fetch all and filter in memory). Still, for a moderate number of photos, this is feasible. - **Cloudflare D1:** For more structured querying (e.g. "filter by state" or search by title), using D1 is cleaner. We could create a D1 table `Media` with columns (id, name, type, size, state, lat, lng, title, description, uploadedAt, etc). On upload, insert a row into this table. Later, `GET /api/v1/media/list` can query this table with filters (SQL queries) to implement the gallery filtering features (state, date range, tags, etc.).

For the immediate deployment, KV could work as a quick solution (since Phase 4 used mock data and localStorage fallback [28]). However, for long-term extensibility, we recommend planning a migration to D1 for media metadata to fully support filtering and search. (This can be done once D1 is set up – and it won't affect how files are stored in R2.)

1. **Response:** Return a JSON response confirming the upload. For example, the Worker can return:

```
{
  "success": true,
  "file": {
     "id": "<media-id>",
     "name": "<original filename>",
     "url": "<public or signed URL to access file>",
     "type": "<mime type>",
     "size": <bytes>,
     "location": { "state": "...", "coordinates": { "lat": ...,
"lng": ... } },
     "uploadedAt": "2025-07-31T23:50:00Z"
```

```
  }
}
```

This mirrors the expected structure from the design docs [29] [30]. Notably, the `url` field can be set to where the frontend should retrieve the media. We have choices for the URL:

2. We could provide a **direct R2 public link**, but since our bucket is private, that would require making the object public or generating a signed URL.
3. Easiest is to provide our own Worker route: e.g. `https://api.awhittlewandering.com/media/<id>`. Then the frontend can use this URL to load the image, and the Worker will internally fetch from R2. We will do this approach for simplicity.
4. Alternatively, we could generate a short-lived signed URL to R2 (with an expiration) and return that. This is more complex but adds security if needed. Given the app's current scope, using the Worker as a proxy is fine; signed URLs can be a later enhancement (we mention it below).

The upload endpoint will enforce **CORS** and size limits: - We will allow CORS from the admin frontend origin (Pages domain or custom domain) on this route, as already configured in the Worker (the Hono app uses CORS for specific origins [31] [32]). This ensures the browser can POST the file via XHR/Fetch. - Cloudflare Workers have a 100MB request limit. This is plenty for images and short videos. We should instruct the admin to keep media files reasonably sized. If needed, we can implement client-side resizing for huge images or chunking for larger videos (though chunking would require a different approach, possibly using the R2 multipart API or streaming). - We might implement basic validation (e.g. only allow certain file types or sizes in the Worker).

Finally, note that in Phase 4 the backend was returning **placeholder responses** and not yet storing to R2 (just mocking success) [28]. The above steps outline the actual implementation to replace those mocks. Cloudflare R2 integration is explicitly the next step [33], and our plan fulfills that.

`GET /media/:key` – Serving Media Files

This endpoint allows retrieval of the uploaded media by its key/ID. It will be used to display images in the frontend (e.g. in the gallery or timeline). We will set it up as a straightforward Worker pass-through: - The client (browser) will request an image URL like `https://api.awhittlewandering.com/media/abcd1234` (where `abcd1234` is the media ID or key). - The Worker receives the request at the `/media/:key` route. It will not require full authentication for GET if we intend to show media on the public site; however, we will secure it in one of two ways: 1. **Authenticated Access:** Require an admin auth token or session for media as well. This means only a logged-in admin can actually retrieve the images. This is secure, but it complicates image loading in the browser (since `<img>` tags don't send auth headers by default). It could work if the admin session is a cookie on the same domain which would be sent automatically. 2. **Unguessable URLs (semi-public):** Treat the media URLs as secret by virtue of randomness. If our media keys are UUIDs or hashes, it's extremely unlikely someone could guess a valid URL. We can allow direct GET without auth, trusting that only those with the exact URL (the admin app) will access it. (We should still restrict CORS so others can't hotlink our images easily.) 3. **Signed URL approach:** As mentioned, we could implement an HMAC signature in the URL query string that the Worker validates [2]. For example, the frontend asks the API for a signed URL for a given media ID, the Worker returns a URL like `/media/abcd1234?sig=<token>&exp=<time>`. The Worker then validates that token on each request (using a

secret key). This ensures even if someone finds the URL, it expires soon. This is a robust approach used in Cloudflare's reference architecture [2], but it's an optional enhancement given the timeline.

- For initial deployment, we can adopt approach (2): use unguessable IDs and restrict knowledge of them to the admin app. We will **enforce that the response is only fetched by allowed origins**. The Worker's CORS policy should allow our front-end origin to GET `/media/*`, and we can deny other origins. This prevents other sites from embedding our images.

Serving the file: The Worker will take the `:key` from the URL path and do:

```
const object = await env.MEDIA_BUCKET.get(key);
if (!object) {
  return new Response("Not found", { status: 404 });
}
return new Response(object.body, {
  status: 200,
  headers: {
    "Content-Type": object.httpMetadata.contentType || "application/octet-
stream",
    "Cache-Control": "public, max-age=31536000"  // perhaps cache images for 1
year at edge/browser
  }
});
```

This will stream the file from R2 to the client. We can set a long Cache-Control header because media (once uploaded) is immutable – this allows Cloudflare's edge cache and the browser to cache the image. If we ever update an image, we'd likely give it a new key or purge it, so caching is safe and improves performance.

Permissions: To reiterate, we'll initially require admin auth for uploading and listing media. Viewing media (GET) can be left open to anyone with the link, which is acceptable if the site eventually displays images to the public. If these images are meant to be private, we would implement the signed URL method or same-site cookie approach. Since Phase 4 is admin-focused, we can treat all media as private-by-default now, and later decide which to publish.

CORS for Media: We will update the Worker's CORS middleware to allow GET requests to `/media/*` from our frontend origin. The code already uses a wildcard route for CORS on all endpoints [31], including GET, so it likely covers `/media/:key` by default. We'll ensure the allowed origins list includes the Pages domain and custom domain (which it does) [34]. This way, the React app can load images from the Worker subdomain without issues.

Summary: With this setup: - Admin can upload images through `/api/v1/media/upload` (goes to R2, metadata to KV/D1). - The app can list media via `/api/v1/media/list` (which the Worker will implement to read from KV/D1 and support filtering by state, etc., as described in Phase 4 docs). - Images/videos are retrieved via `/media/:key` URLs. We might embed these in the frontend gallery or even generate thumbnails later. - Future enhancements: We noted the plan to add **thumbnail generation** and **image optimization** [35]. Cloudflare offers an *Images* service or we could use Workers to resize. For now, we'll store

full images. In the future, we might generate a smaller thumbnail on upload (either on the client or using a Cloudflare Image Resizing request) and store it alongside the original in R2 (perhaps with a `thumb-<id>` key). Our `/media/:key` route could detect if a `?thumb=true` param is requested and serve the thumbnail instead or dynamically transform the image (Cloudflare's image resizing API could be invoked in Worker fetch). These are considerations for later; initial deployment can serve original images given the likely low volume.

By leveraging R2 and these endpoints, we adhere to a fully **Cloudflare-native solution for media**. No external storage is needed, and the Worker controls access. This approach aligns with Cloudflare's recommended serverless image management (store blobs in R2, store metadata in KV/DB, secure access via signed links or origin checks) [2].

CI/CD Pipeline (GitHub Actions for Pages & Workers)

To streamline deployments, we will use **GitHub Actions** to build and deploy both the frontend and backend, with separate workflows for the **staging** environment and **production** (main branch). The goals of our CI/CD setup are:

- Automate the build/test process on every commit.
- Deploy to a **staging** environment for any updates merged into a `staging` branch (or we could use a specific GitHub environment). The staging deployment will target Cloudflare Pages (frontend) in preview mode or a separate Pages project, and a staging Worker instance (perhaps on a workers.dev subdomain or a reserved route).
- On merging to **main**, automatically deploy to production (the live Pages site and the live Worker on the custom domain).

Workflow Structure:

1. **Build & Test (build.yml):** We will have a workflow that runs on pull requests or pushes, which installs dependencies and runs tests/linters for both frontend and backend. This ensures code quality before deployment. For example:
2. Use Node 18.x in the job.
3. `npm install` (or `npm ci`) in both `frontend/` and `backend/edge-worker/` folders.
4. Run `npm run build` in each to ensure they compile.
5. Possibly run any unit tests (e.g. `npm t`) especially for the data aggregator logic once tests are added.
6. This workflow doesn't deploy, it just checks the build passes.
7. **Deploy to Staging (deploy-staging.yml):** Triggered on push to the `staging` branch.
8. **Frontend Pages:** Use Cloudflare's **Wrangler** CLI or the Pages GitHub Action to deploy the static build. We'll generate a production build of the React app (`npm run build` in `frontend/`) which outputs to `frontend/dist` (assuming Vite output dir). Then use `wrangler pages publish`

`frontend/dist --project-name=<Pages Project Name>` to upload to Cloudflare Pages **staging** project or environment.

- We might set up two Pages projects: one for production (bound to main branch or manual deploys) and one for staging (bound to staging branch). Alternatively, Cloudflare Pages supports preview deployments on every commit automatically. However, to have a persistent staging site with custom URL, a separate Pages project is useful. We can name it something like "AWhittleWandering Staging" which might deploy at a URL like `staging.awhittlewandering.pages.dev` or a subdomain of our site.

9. **Backend Worker:** Deploy the Worker using Wrangler. We can use Cloudflare's official action `cloudflare/wrangler-action@v3` [36] . The action needs the Cloudflare API token and account info (we'll store those in GitHub Secrets). We can run `wrangler deploy --env staging` to publish the Worker to a staging environment. In `wrangler.toml`, we'll define a `[env.staging]` with a `workers_dev` route (like `awhittlewandering-api-staging.<your-subdomain>.workers.dev`) or even map it to a subdomain (like `staging-api.awhittlewandering.com`). This way, the staging frontend can call the staging API.
10. **Environment Variables:** During this staging deploy, we'll inject the appropriate environment secrets (see below). For example, the Pages build needs the API base URL set to the staging API URL. We can achieve this by defining an environment variable `VITE_API_BASE_URL` in the workflow (or in Cloudflare Pages project settings for the staging project). Similarly, Mapbox token and any other public config should be provided. On the Worker side, Wrangler will pick up secrets that we've set (Tessie API key, etc.) if those are already configured in the Cloudflare environment. If not, we can have the action run `wrangler secret put` commands (with values from GH Secrets) for the staging environment before deploying.

11. After deployment, we should have a functional staging site – useful for QA and for new contributors to see their changes in action before going to prod.

12. **Deploy to Production (deploy-prod.yml):** Triggered on push to `main` branch (and also maybe manually via a workflow dispatch).

13. This will mirror the staging deploy but target production:
14. Frontend: Use Wrangler or Pages action to deploy to the production Pages project (which is likely the one backing awhittlewandering.com). If Cloudflare Pages is connected to GitHub, it might auto-deploy main by itself; but since we want to include backend deployment in the same pipeline, using Wrangler's direct upload is fine. For example: `wrangler pages publish frontend/dist --project-name=awhittlewandering` for production.
15. Backend: `wrangler deploy` for the main environment (or no env, using default). In `wrangler.toml` default, we'll set the custom domain route for production (like `route = awhittlewandering-api.example.com/api/*` if using a domain, or just rely on workers.dev subdomain given).
16. Ensure secrets are in place (Tessie API key for prod, etc.). We might configure those ahead of time via wrangler CLI so they persist; the GitHub Action can assume they're already there. Or we can automate it similarly to staging.

17. We will also differentiate any environment-specific values:

    - For example, the frontend `VITE_API_BASE_URL` should point to the production API domain (`https://awhittlewandering-api.<workers>.dev` or custom domain). We can set this via GH Secrets or in Pages settings for production.
    - If using any analytics or other keys that differ between staging/prod, handle those as well.

18. **Secrets Management in CI:** We will add the following to the repository's GitHub secrets:

19. `CF_API_TOKEN`: a Cloudflare API token with permissions to deploy Workers and Pages.
20. `CF_ACCOUNT_ID`: our Cloudflare account ID.
21. `CF_PROJECT_NAME`: (if using direct Wrangler deploy to Pages) the name or ID of the Pages project.
22. `VITE_MAPBOX_TOKEN`, `VITE_TESSIE_API_KEY`, etc.: Actually, for security, we **do not** want to store Tessie API key as a VITE variable (since that would end up in the frontend). Tessie key should be kept in Worker secret only. So, we'll store `TESSIE_API_KEY` (no VITE prefix) in GH Secrets and use it to set the Worker secret.
23. Mapbox token can be stored as `MAPBOX_TOKEN` in GH Secrets and passed to the Pages build. Mapbox tokens are public (but we still don't commit it).
24. Other keys: e.g. `OPENWEATHER_API_KEY` if we integrate weather, `JWT_SECRET` if we have one for auth, etc., should be similarly handled.
25. Admin credentials: If the admin login uses a password or JWT secret, that should be set as a Worker secret as well (e.g. `ADMIN_PASSWORD` or a hashed version).

The CI workflows will export these secrets as environment variables for the build steps. For example:

```
- name: Build frontend
  run: npm run build
  working-directory: frontend
  env:
    VITE_API_BASE_URL: ${{ secrets.STAGING_API_BASE_URL }}
    VITE_MAPBOX_TOKEN: ${{ secrets.MAPBOX_TOKEN }}
```

This ensures the built frontend has the correct API endpoint and Mapbox token injected. The Tessie key is **not** provided to the frontend build. Instead, in the Worker deploy step, we might run something like:

```
- name: Publish Worker
  uses: cloudflare/wrangler-action@v3
  with:
    apiToken: ${{ secrets.CF_API_TOKEN }}
    environment: 'staging'
    command: publish  # (or deploy)
```

We'll rely on the Worker's environment configuration (in wrangler.toml) which references secrets set earlier. If needed, we can also automate secrets:

```
- name: Set Tessie Secret
  run: npx wrangler secret put TESSIE_API_KEY --env staging --value $
{{ secrets.TESSIE_API_KEY }}
```

(This would execute prior to the publish, if the secret isn't already present on Cloudflare.)

1. **Branch Protections & Flow:** We will use `staging` for integration testing. Developers create feature branches, merge into `staging` for testing on the staging environment, then finally merge to `main` for prod. The CI ensures each step is verified. This separation prevents accidental deployments to prod and gives a safe space to test new features (like verifying the unified-data endpoint or media upload in a staging environment with real Cloudflare services).

2. **Monitoring Deployments:** We can utilize GitHub Actions job outputs or Slack notifications to inform when a deploy goes live. Cloudflare's action also supports marking the GitHub deployment status. This will help maintain visibility. Additionally, since Cloudflare Pages and Workers provide logs, we can verify post-deploy that everything started correctly (for example, use `wrangler tail` or the Cloudflare dashboard to check logs after a production deploy).

Overall, the CI/CD setup will provide a **one-click deployment** experience: pushing to the main branch results in an updated website and API within minutes, with all secrets and environment differences handled. The repository README will be updated to document this workflow for contributors.

Secrets Management and Configuration

Maintaining **secure separation of secrets** is crucial. The frontend should **never expose sensitive API keys** (like Tessie). Here's our plan for managing configuration:

- **Tessie API Key:** This will be stored as a secret in the Cloudflare Worker environment (accessible via `env.TESSIE_API_KEY`). It will **not** be present in the client-side code. In local development, we'll set this in `wrangler.toml` [vars] or via `wrangler secret put`. In production, it's configured via CI or Cloudflare dashboard. The frontend will only communicate with our Worker, which uses the Tessie key internally.

- **Mapbox Token:** Mapbox requires a public token for the web map. This is less sensitive but still should be configurable. We'll supply it to the frontend as a build-time environment variable (e.g. `VITE_MAPBOX_TOKEN`). The token will be included in the frontend bundle (e.g. used when calling `mapboxgl.accessToken = "<token>"`). We'll restrict the token's usage to our domain in Mapbox settings for safety. This token is stored in our Pages project configuration or GH Secrets (not hard-coded). The README will instruct to obtain a Mapbox token and set `VITE_MAPBOX_TOKEN` in the `.env`.

- **API Base URL:** The frontend needs to know where to reach the Worker API. During development it might be `http://localhost:8787` (if running `wrangler dev`), in staging it might be a workers.dev URL, and in production the custom domain. We use an environment variable `VITE_API_BASE_URL` for this [37]. For example, in `.env` we might have:

```
VITE_API_BASE_URL="https://awhittlewandering-api.kd8jc7v8cd.workers.dev"
```

as shown in the project config [38]. We'll update this to the final domain when known. This variable is injected at build time, so the frontend's fetch calls hit the correct endpoint.

- **OpenWeather API Key (future use):** The project mentions integrating OpenWeather for weather data. If/when we use it, that key will also be kept in the Worker (since we likely call weather API from the server to include weather info in telemetry). It would be set as `OPENWEATHER_API_KEY` in Worker env (as in README) [39]. The front end doesn't need direct access to it.

- **Admin Credentials:** The admin login likely uses either a password or an OTP. In Phase 4 testing, a password was hardcoded for convenience [40]. For production, we will configure an **environment variable for admin password or a hashed password** that the Worker can check. Even better, we can implement a JWT auth: the Worker holds a `JWT_SECRET` and on login, generates a token. That secret stays in the Worker config. The frontend only gets the token, which it stores and sends with requests. All of these secrets (password hash, JWT secret) will be stored via Wrangler secrets.

- **JWT vs Session:** If we choose JWT for admin auth, the token is stored in client (probably localStorage) and sent in headers. If we choose a session cookie, the Worker might use a KV to store session state keyed by a cookie value. In either case, secrets (JWT signing key or session encryption key) remain server-side.

- **Environment-specific Settings:** We may have slight differences between staging and prod (like using a test Tessie vehicle vs real). For example, we could use a **sandbox Tessie key** in staging if available (or the same key but possibly hitting a different vehicle?). If only one vehicle, we'll just use the same key but be mindful of not driving the car crazy with queries in staging. In any case, the separation is easy to manage via Wrangler envs:

- `TESSIE_API_KEY` in `[env.production]` vs another in `[env.staging]` if needed.
- `VITE_API_BASE_URL` differing in the frontends.

- **Documentation:** We will update the README's **Environment Configuration** section to clearly delineate what needs to be set. Likely something like:

```
# .env (frontend)
VITE_API_BASE_URL=https://<your-prod-worker-domain>
VITE_MAPBOX_TOKEN=<your_mapbox_public_token>

# Cloudflare Worker secrets (via wrangler.toml or dashboard)
TESSIE_API_KEY=<your_tessie_api_key>
OPENWEATHER_API_KEY=<your_openweather_key>
JWT_SECRET=<some_random_secret_for_admin_tokens>
```

The current README already shows a split between frontend and backend config [38]. We will ensure it's up-to-date (for example, removing any `VITE_TESSIE_API_KEY` usage which might have existed – that is no longer needed on the frontend). Instead, emphasize the Tessie key goes in the Worker config only.

By strictly isolating secrets, we prevent any sensitive info from leaking to the user's browser. The frontend operates with only the minimal info (Mapbox token and API URLs). All heavy lifting with keys happens in the Worker. This also means **rotating keys** (if needed) is easier: e.g. update the secret in Cloudflare without changing the frontend.

Data Validation, Fallbacks, and Testing

To ensure a robust user experience, we will implement validation and fallback behavior both in the backend and frontend:

Schema Validation: We will define TypeScript interfaces (and possibly Zod schemas) for the unified data response and other API payloads. The project already uses shared types and mentions Zod for runtime validation [41] [42]. On the Worker side, after assembling the unified data object, we can validate it against a schema to catch any missing fields or incorrect types before sending it out. This adds a safety net in case Tessie's data format changes or some field is undefined. For example, we can create a Zod schema for `UnifiedJourneyData` that requires all the expected fields. If validation fails, we log an error (and perhaps still return what we have with an `incomplete: true` flag or an error field). This way, the API won't silently return malformed data.

On the frontend side, the custom React hooks (`useUnifiedJourneyData`, etc.) will also safely handle the data: - The hook already sets a loading state and catches errors [25] [43]. We will keep the loading skeleton UI in place (so the dashboard shows a placeholder while waiting for data). - If an error occurs (e.g. network failure or 500 from server), the hook sets `error`, and components like AdventureHero display an error message banner instead of breaking the UI [44] [45]. We should ensure these error messages are user-friendly (e.g. "Live data is currently unavailable. Retrying..."). - We might implement a **retry/backoff** strategy in the hook: right now it polls every 30s regardless [22]. If errors persist, we could slow down the polling or stop after a few failures and show a "reconnect" button for the admin to manually resume. This prevents spamming the server or Tessie if something is wrong (though 30s interval is quite reasonable).

Live Telemetry Indicator: We want the UI to clearly indicate when data is "live" versus stale. We can use the `tessieStatus` from the unified data for this: - If `tessieStatus.connected` is true, and `lastSync` is very recent (within, say, one polling interval), then we consider telemetry live. The UI can show a green "LIVE" indicator (for example, a pulsing dot with "Live" text). In fact, the design already shows a green dot labeled "LOCATION: Connecticut" in the hero when data is live [46]. We will maintain that. - If `connected` is false or lastSync is old, we change the indicator (maybe grey dot or "Offline" message). This way, viewers know the data might not be updating in real-time. For the admin, this is a clue to check Tessie or connectivity.

We will also display the last update time. In the hero component code, they show "LAST UPDATE: {time}" [47]. We will ensure that continues to function, using `currentStatus.lastUpdate` or

`tessieStatus.lastSync` – whichever is more appropriate – to populate that. This gives transparency on data freshness.

Fallback Data Rendering: In case the unified-data endpoint fails entirely (e.g. returns no data), we have a couple of fallback options: - Use **cached data in the browser**: We can have the frontend store the last successful response (perhaps in `localStorage`). Phase 4 mentioned a localStorage fallback for media data [28]; we can apply similar for telemetry. For example, whenever a new unified data is fetched, save it to localStorage. If the app loads and the API call fails, we can load that cached data so at least something is shown (with a warning that it's not live). This is an optional improvement for resilience. - Use **mock data** as a last resort: For development or demo mode (when Tessie isn't connected), we maintain a sample JSON that can be returned. Possibly triggered by a flag, if no real data, the Worker could return a hardcoded sample that matches our schema (like the example with Connecticut, 29 states, etc. that was used in legacy endpoints [48] [49]). This ensures the UI is populated for demos. In production, we'd only do this if Tessie is completely unreachable and we want to show at least the route history or something.

Frontend Error Guards: We will audit the frontend components to ensure they check for presence of data before rendering. For instance, using optional chaining or conditional rendering until data is available. The `useUnifiedJourneyData` hook returns `overview`, `currentStatus` etc. or `null` if not yet loaded [50]. Components should handle `null` gracefully (e.g. show "Loading..." or just blank fields). Many parts of the UI already do this by checking `loading` or using default values (for example, in AdventureHero, they default to placeholder values if `journeyInsights` is empty [51]). We will continue this practice: - If `statesVisited` is not available, show "–" or a default. - If current location is missing, show "Unknown" or last known state. - Wrap any dynamic lists (timeline, media gallery) in a condition so that if data is not there or an error occurred, the app won't crash. Possibly render a message like "Timeline data unavailable."

Unit Tests for Aggregation Logic: A key part of this deployment is confidence that our unified-data assembly works correctly. We will write unit tests for the backend aggregator function: - Provide sample inputs like a Tessie vehicle state JSON, a list of drives JSON, etc. (We can use the Tessie API docs' examples [52] [53]). - Test that the output of our combine function produces the expected `JourneyOverview.daysElapsed`, `totalMiles`, etc., and that it correctly counts states visited given a set of drive locations. - Test edge cases: no drives (just starting trip), or car offline (simulate Tessie state missing). - Test that timeline entries map correctly from Tessie format to our format (we have mapping guidelines already defined for drives and charges [19] [54]). - If we use Zod for validation, we'll test that invalid data is caught by the schema.

These tests can run in CI with `npm test` to prevent deploying a broken aggregator.

Manual Testing & Verification: In addition to automated tests, after deploying to staging, we will do thorough manual testing: - Verify that the frontend connects to the staging worker and displays live data. For example, if the car is on or moving, see that battery or location updates on the dashboard. - Simulate the car being offline (maybe turn off polling or use a wrong Tessie key) and confirm the frontend shows a disconnect indicator or error. - Test uploading media on staging: select an image, upload, ensure it appears in the gallery with correct metadata. Then check that the image can be retrieved via the `/media/:key` URL (perhaps by opening it in a new tab or checking network panel). - Check that unauthenticated requests to upload or list media are rejected (try calling the API without token, expect 401). - Test the rate limit: possibly script rapid calls to `/unified-data` and ensure after a threshold it responds with 429 or slows down.

Frontend Fallback Display: We might add a small banner in the UI that only appears if data hasn't updated in a while. For instance, if `lastSync` is older than 2 minutes, show "Data is temporarily stale. Last update X minutes ago." This sets the expectation for viewers. The admin can also be given a manual "Refresh now" button that forces a fetch (bypassing the interval) – useful if they think data got stuck.

Logging and Monitoring: We will use Cloudflare Workers' logging (`console.log`) to record key events (like "Fetched Tessie data, battery=80%, location=CT"). This will help in debugging issues in production by checking the Worker logs (`wrangler tail`). We can also monitor Tessie API responses for any error messages and log them. If Tessie has rate limit responses, log those with severity so we know to adjust our polling or caching further.

By implementing these validation and fallback measures, we ensure the app remains **reliable** even under adverse conditions (API downtime, network issues, etc.), and that contributors can trust the system to catch errors early (through tests and schema checks).

Deployment & Debugging Guide for Contributors

This section provides a step-by-step guide for new contributors to deploy and troubleshoot the app in the Cloudflare environment:

Local Development Setup

1. **Prerequisites:** Ensure you have Node.js 18+ (or use Bun if preferred) and Cloudflare's Wrangler CLI installed. Also, obtain the required API keys:
2. Tessie API Key (from the Tessie account connected to the Tesla).
3. Mapbox API Token.
4. (Optional) OpenWeather API Key (if testing weather features).

5. Cloudflare Account - If you are a maintainer, you should have access to the Cloudflare account for this project, or use the provided development credentials.

6. **Clone & Install:** Clone the repository and install dependencies:

   ```
   git clone https://github.com/JW-Flo/AWhittleWandering.git
   cd AWhittleWandering
   npm install
   ```

   This will install both frontend and backend dependencies (the project might be set up as a monorepo or separate, but a root install should cover both given a common package or workspaces).

7. **Configure Environment:** Create a `.env` file in the `frontend/` directory with the following (adapt with your dev values):

```
VITE_API_BASE_URL="http://localhost:8787"
# if using wrangler dev on port 8787
VITE_MAPBOX_TOKEN="<your_mapbox_token>"
```

And configure Wrangler for the backend. In `backend/edge-worker/wrangler.toml`, set:

```
vars = {
   TESSIE_API_KEY = "<your_tessie_key>",
   OPENWEATHER_API_KEY = "<your_openweather_key>"
}
# (Alternatively, use `wrangler secret put` for these)
```

Also check `wrangler.toml` for KV and R2 bindings; if not present, add the KV namespace and R2 bucket binding names you're using for dev (you might use the Cloudflare dashboard to create dev instances of KV/R2 or use Miniflare for local testing of KV/R2).

8. **Run Development Servers:** In two terminals, start the frontend and backend:

9. Frontend: `npm run dev --workspace=frontend` (or `cd frontend && npm run dev`). This starts the Vite dev server (likely on http://localhost:5173 or similar, as configured).
10. Backend: `npm run dev --workspace=backend/edge-worker` (or `cd backend/edge-worker && wrangler dev`). This runs the Cloudflare Worker in development mode on http://localhost:8787 by default [55]. The dev mode will proxy requests to your local code and allow debugging. Ensure it says "Listening on 8787" or adjust the port accordingly.

The frontend will proxy API calls to the backend. If the frontend dev server is on a different port (5173), you may need to configure Vite to proxy `/api` calls to `localhost:8787`. Check `vite.config.js` or the network requests; if calls to `/api/v1/unified-data` fail due to CORS, you can either set `VITE_API_BASE_URL` to the full URL (including `localhost:8787`) or configure a proxy in dev.

1. **Log in and Test:** Open the app (http://localhost:5173 if that's the front) in your browser. You should see the homepage/dashboard. If you have live Tessie credentials and the car is active, you may start seeing real data populate (location, battery, etc.). If not, the app might show error or placeholder. You can use the provided admin credentials to log into admin mode (if configured). The Phase 4 default password was `RoadTrip48States!2025` [40] – if unchanged, try that. Otherwise, consult environment for the admin password.

Test that: - The unified data is fetching (open dev console Network tab to see requests to `/api/v1/unified-data` and their responses). - The timeline and stats populate (or at least no errors). - Media tab (if present): try uploading a sample image. Since in dev we might not have Cloudflare R2 accessible, the upload might be faked or you need to configure Miniflare to simulate R2. You can skip actual R2 in local dev and trust it works in staging, or test with a smaller image and see if any errors logged.

If something is not working, check both terminals: - **Frontend logs:** look for any React error or console.log output. If an error like "Failed to fetch" appears, it could be CORS or wrong URL. - **Backend logs:** Wrangler

dev will show console.log and errors from the Worker. If there's an exception in the unified-data route, it will print a stack trace here. Use this to debug (e.g. missing a secret, or a fetch to Tessie failing).

1. **Troubleshooting Common Issues:**
2. *CORS errors in dev:* Ensure the Worker dev is allowing the origin of the Vite dev server. You may add `http://localhost:5173` to the CORS allowed origins in the code [56] if it's not already. Restart wrangler dev after changes.
3. *Tessie API not responding:* If using a real Tessie key, ensure the vehicle is accessible (maybe wake it via Tessie app). If you get unauthorized, double-check the key. Use `wrangler tail` even in dev to see any logs (wrangler dev might show logs too).
4. *KV/R2 in dev:* Miniflare (the dev environment) might not have your KV namespace or R2 bucket by default. You can configure it via `wrangler.toml` dev settings or use `--kv` flags. If KV get/put calls error out in dev, that's likely why. For simplicity, you can bypass KV caching in dev (since one user anyway). Similarly, R2 might not function without a binding – consider catching those calls in dev or using a local file fallback.

Deploying to Cloudflare Staging

Once features are tested locally, you can push to the staging branch to test on Cloudflare's infrastructure: 1. **Git Workflow:** Commit your changes and open a PR to merge into `staging`. Once merged, the GitHub Action will run. Monitor the Actions tab for the staging deploy workflow. It should build the app and then publish to Cloudflare. If it fails, see the logs for the step that failed (could be build error or Cloudflare API issue). 2. **Staging Verification:** Access the staging site URL. If we set up a staging domain (e.g. `staging.awhittlewandering.pages.dev` or `staging.awhittlewandering.com`), go there. Verify the site loads and is using the staging API (you can check network calls to see if they go to a staging workers.dev URL). - Test basic functionality again (data fetching, etc.) on staging. This is almost identical to local, but now using Cloudflare's edge. This step may catch issues like missing environment variables on Cloudflare (for instance, if Tessie key wasn't added to the staging Worker environment, the unified-data endpoint might return an auth error). If something is off, you may need to add those via `wrangler secret` or the Cloudflare dashboard, then re-deploy. - Check the Cloudflare Worker logs for staging by running `wrangler tail --env staging` (if configured) or via the Workers dashboard. This helps see if Tessie calls succeeded, etc. 3. **Media on Staging:** If feasible, test uploading an image on staging. This will actually go to your Cloudflare R2 bucket. After upload, use the Cloudflare R2 browser (in your account dashboard) to see if the file appeared in the bucket. Also test viewing it via the provided URL. This ensures R2 permissions are correct and the endpoint works.

Deploying to Production

When everything looks good on staging: 1. Merge the staging branch into `main`. This triggers the production deploy workflow. Again watch the Actions logs. On success, it means Cloudflare Pages updated the live site and the Worker was updated. 2. Visit the production site (awhittlewandering.com). It should be running the new code. Verify the API calls are going to the production Worker (which might be a custom domain or the workers.dev if still using that). Ensure data loads properly. 3. Quick regression test: check that public-facing features (live map, stats display) work for a normal viewer. Then log in to admin (if applicable on production) and verify admin-only features (upload, etc.) function as expected. 4. After a successful deploy, monitor things over time. Use the health endpoint (`/health`) which should return

`"status":"ok"` [57] to confirm the API is up. You can also enable Cloudflare Alerts or logs if desired to catch any runtime errors.

Debugging in Production

If issues arise in production: - Use `wrangler tail` for the production worker to stream logs in real-time. You can filter by status codes or strings if needed. For example, to watch for errors: `wrangler tail | grep "ERROR"`. - Check Cloudflare Analytics for Workers (in the dashboard) – it can show if there are spikes in errors or if your requests are being rate-limited. - If the issue is visual or data-related on the frontend, use browser devtools. Verify the network responses (did `/unified-data` return what we expect? any error message in it?). Open the console for any JS errors. - For media, if an image isn't showing, check its URL and try to fetch it manually (maybe the key is wrong or the R2 bucket didn't have CORS and the request is blocked – the console would show CORS errors in that case). - Sometimes a fresh deploy might have caching issues (the browser might have cached old JS). Do a hard refresh or clear cache. Cloudflare Pages usually appends a hash to filenames to avoid this, though. - If a secret was misconfigured (e.g. Tessie key missing), the Worker might have thrown an exception on fetch. The logs would show a 401 from Tessie or similar. In that case, add/fix the secret and redeploy the Worker.

Documentation and Support

New contributors should read the updated **README** for environment setup and refer to this guide for architecture understanding. It's also helpful to read the API docs in `docs/` for specifics on data formats (e.g. API_REFERENCE.md and API_DOCUMENTATION.md already detail Tessie and our API data models). The project's commit history and issue tracker (if used) may have context on why certain decisions were made (like the unified-data approach).

Encourage contributors to use staging for any experiments (so as not to disrupt the live trip tracking). We can also set up an "admin sandbox" mode on staging if needed (e.g. using a different Tessie vehicle or static data) to allow UI tweaks without affecting real trip data.

Finally, emphasize a mindset of **security** and **extensibility**: - Do not commit any keys to repo. Use env variables as described. - Test changes with both admin and public perspective. - Keep an eye on Cloudflare limits (Tessie rate limits, Worker CPU time, R2 read/writes) and optimize as needed (our caching strategy should help). - Document any new environment variable or setup step in the README so the next person knows what to do.

With this deployment plan, the *A Whittle Wandering* Tesla road trip app will be fully migrated to Cloudflare's serverless stack, enabling a secure, scalable, and maintainable platform for the remainder of the 48-state journey and beyond. Enjoy the live telemetry and happy debugging!

Sources:

- Project README – architecture & config [1] [38] [23]
- Frontend Hook – unified data structure & polling [58] [14] [15] [21] [43]
- Phase 4 Summary – new media endpoints and R2 integration plans [27] [33]
- Cloudflare Reference – R2 for images & KV for metadata (link signing) [2]
- API Docs – Tessie endpoints and data mapping [3] [4] [19]

- Existing API Endpoints – legacy trip status (for comparison) [48] [49]

---

[1] [8] [23] [37] [38] [39] [41] [42] README.md
https://github.com/JW-Flo/AWhittleWandering/blob/3c00746511753f73c82c3f86a6b124b91df3a1fc/README.md

[2] Serverless image content management · Cloudflare Reference Architecture docs
https://developers.cloudflare.com/reference-architecture/diagrams/serverless/serverless-image-content-management/

[3] [4] [16] [19] [29] [30] [52] [53] [54] API_DOCUMENTATION.md
https://github.com/JW-Flo/AWhittleWandering/blob/3c00746511753f73c82c3f86a6b124b91df3a1fc/docs/API_DOCUMENTATION.md

[5] [6] [10] [11] [18] [20] [24] API_REFERENCE.md
https://github.com/JW-Flo/AWhittleWandering/blob/3c00746511753f73c82c3f86a6b124b91df3a1fc/docs/API_REFERENCE.md

[7] [44] [45] [46] [47] [51] AdventureHero.tsx
https://github.com/JW-Flo/AWhittleWandering/blob/3c00746511753f73c82c3f86a6b124b91df3a1fc/frontend/src/components/AdventureHero.tsx

[9] [12] [13] [26] [31] [32] [34] [48] [49] [56] [57] index.ts
https://github.com/JW-Flo/AWhittleWandering/blob/3c00746511753f73c82c3f86a6b124b91df3a1fc/backend/edge-worker/src/index.ts

[14] [15] [17] [21] [22] [25] [43] [50] [58] useUnifiedJourneyData.ts
https://github.com/JW-Flo/AWhittleWandering/blob/3c00746511753f73c82c3f86a6b124b91df3a1fc/frontend/src/hooks/useUnifiedJourneyData.ts

[27] [28] [33] [35] [40] [55] PHASE_4_COMPLETION_SUMMARY.md
https://github.com/JW-Flo/AWhittleWandering/blob/3c00746511753f73c82c3f86a6b124b91df3a1fc/docs/PHASE_4_COMPLETION_SUMMARY.md

[36] Deploy to Cloudflare Workers with Wrangler · Actions - GitHub
https://github.com/marketplace/actions/deploy-to-cloudflare-workers-with-wrangler